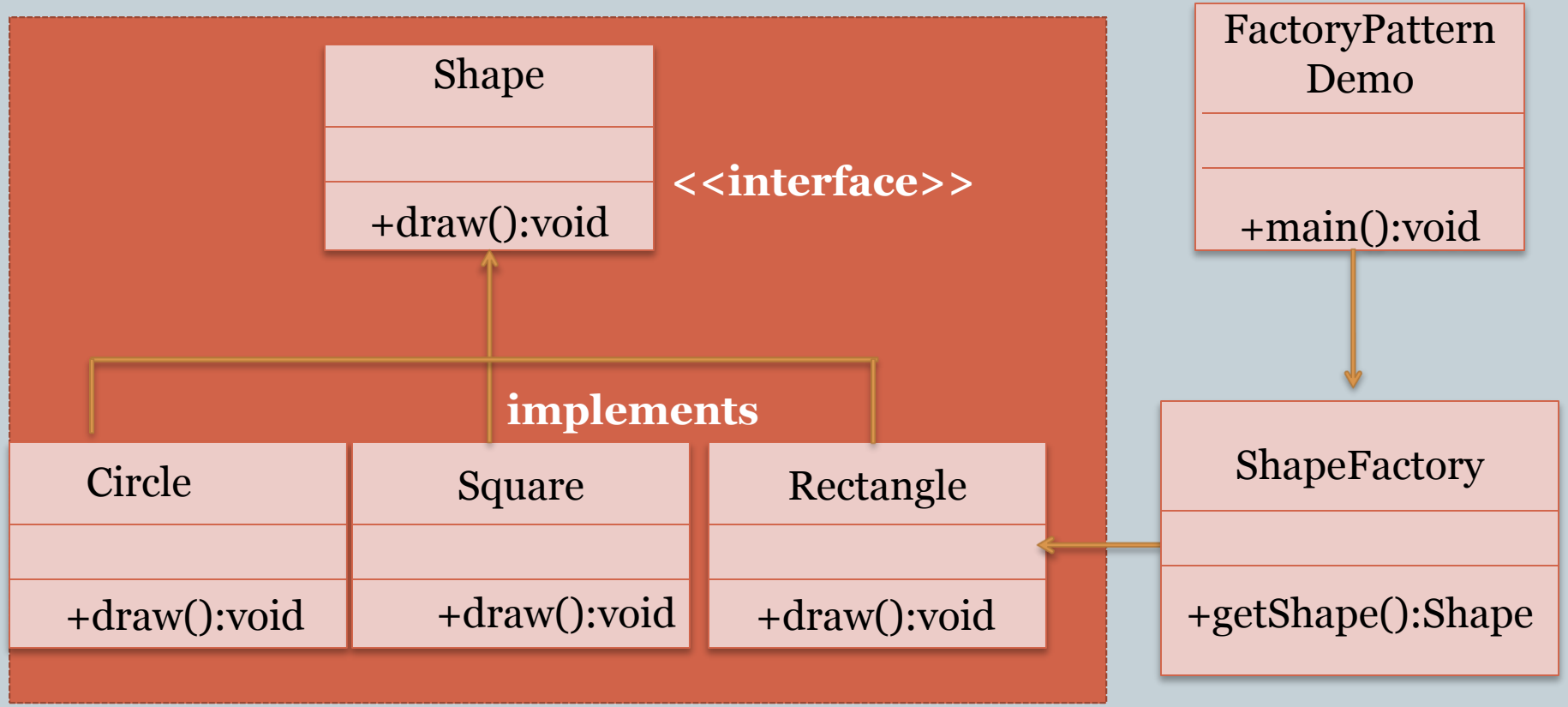
Design Pattern

BY USING THE DESIGN PATTERNS YOU CAN MAKE YOUR CODE MORE FLEXIBLE, REUSABLE AND MAINTAINABLE. IT IS THE MOST IMPORTANT PART BECAUSE JAVA INTERNALLY FOLLOWS DESIGN PATTERNS.

Factory Pattern

- Factory pattern we create object without exposing the creation logic to the client and refer to newly created object using a common interface.
- A Factory Pattern or Factory Method Pattern says that just **define an interface or abstract class for creating an object but let the subclasses decide which class to instantiate.** In other words, subclasses are responsible to create the instance of the class.
- **Advantage of Factory Design Pattern**
- Factory Method Pattern allows the sub-classes to choose the type of objects to create.
- It promotes the **loose-coupling** by eliminating the need to bind application-specific classes into the code. That means the code interacts solely with the resultant interface or abstract class, so that it will work with any classes that implement that interface or that extends that abstract class.
- **Usage of Factory Design Pattern**
- When a class doesn't know what sub-classes will be required to create
- When a class wants that its sub-classes specify the objects to be created.
- When the parent classes choose the creation of objects to its sub-classes.
- In Factory pattern, we create objects without exposing the creation logic to the client and refer to newly created object using a common interface

Class Diagram



Abstract Factory Pattern

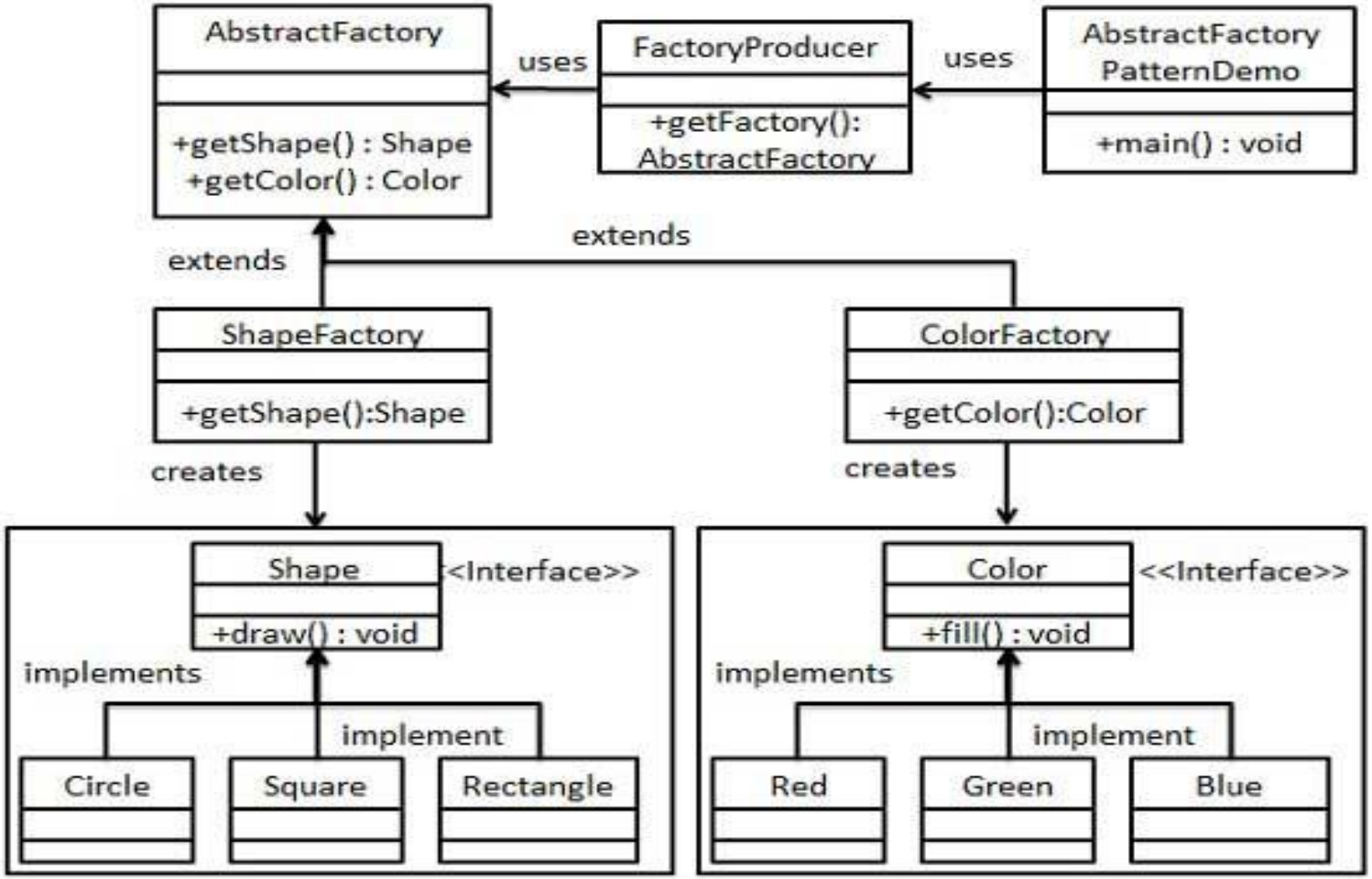
- Abstract Factory patterns works around a super-factory which creates other factories this factory is also called as factories .
- Abstract Factory pattern an interface is responsible for creating a factory of related objects, without explicitly specifying their classes.

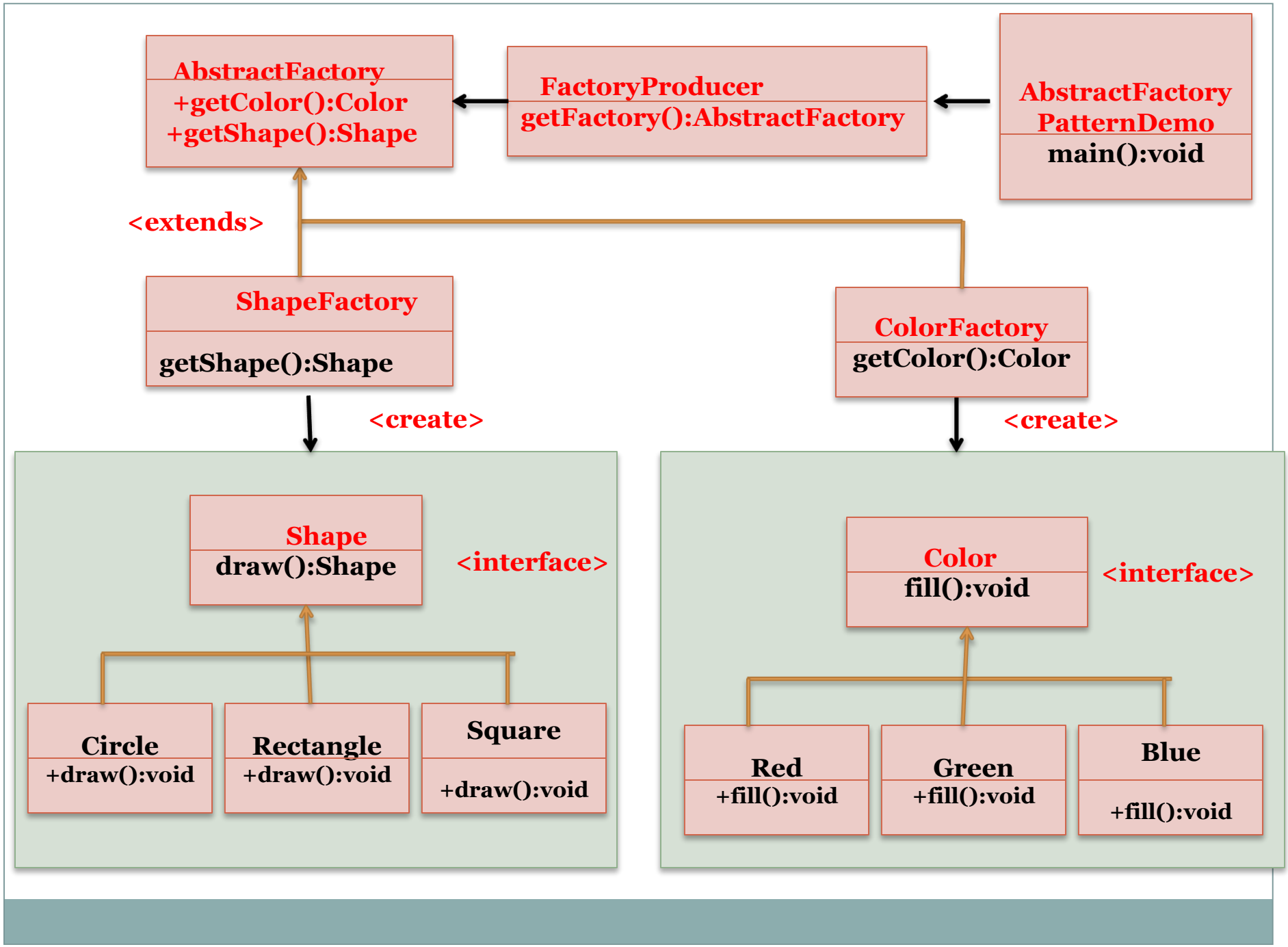
Advantage of Abstract Factory Pattern

- Abstract Factory Pattern isolates the client code from concrete (implementation) classes.
- It eases the exchanging of object families.
- It promotes consistency among objects.

Usage of Abstract Factory Pattern

- When the system needs to be independent of how its object are created, composed, and represented.
- When the family of related objects has to be used together, then this constraint needs to be enforced.
- When you want to provide a library of objects that does not show implementations and only reveals interfaces.
- When the system needs to be configured with one of a multiple family of objects.


AbstractFactory
+getShape() : Shape
+getColor() : Color
uses
FactoryProducer
+getFactory():
AbstractFactory
uses
AbstractFactory
PatternDemo
+main() : void
extends
extends
ShapeFactory
+getShape():Shape
ColorFactory
+getColor():Color
creates
creates
Shape
<<Interface>>
+draw() : void
implements
implement
Circle
Square
Rectangle
Color
<<Interface>>
+fill() : void
implements
implement
Red
Green
Blue


AbstractFactory
+getColor():Color
+getShape():Shape
FactoryProducer
getFactory():AbstractFactory
AbstractFactory PatternDemo
main():void
<extends>
ShapeFactory
getShape():Shape
ColorFactory
getColor():Color
<create>
<create>
Shape
draw():Shape
<interface>
Color
fill():void
<interface>
Circle
+draw():void
Rectangle
+draw():void
Square
+draw():void
Red
+fill():void
Green
+fill():void
Blue
+fill():void

Singleton Design Pattern

Singleton Pattern says that just "define **a class that has only one instance and provides a global point of access to it".**
In other words, a class must ensure that only single instance should be created and single object can be used by all other classes.

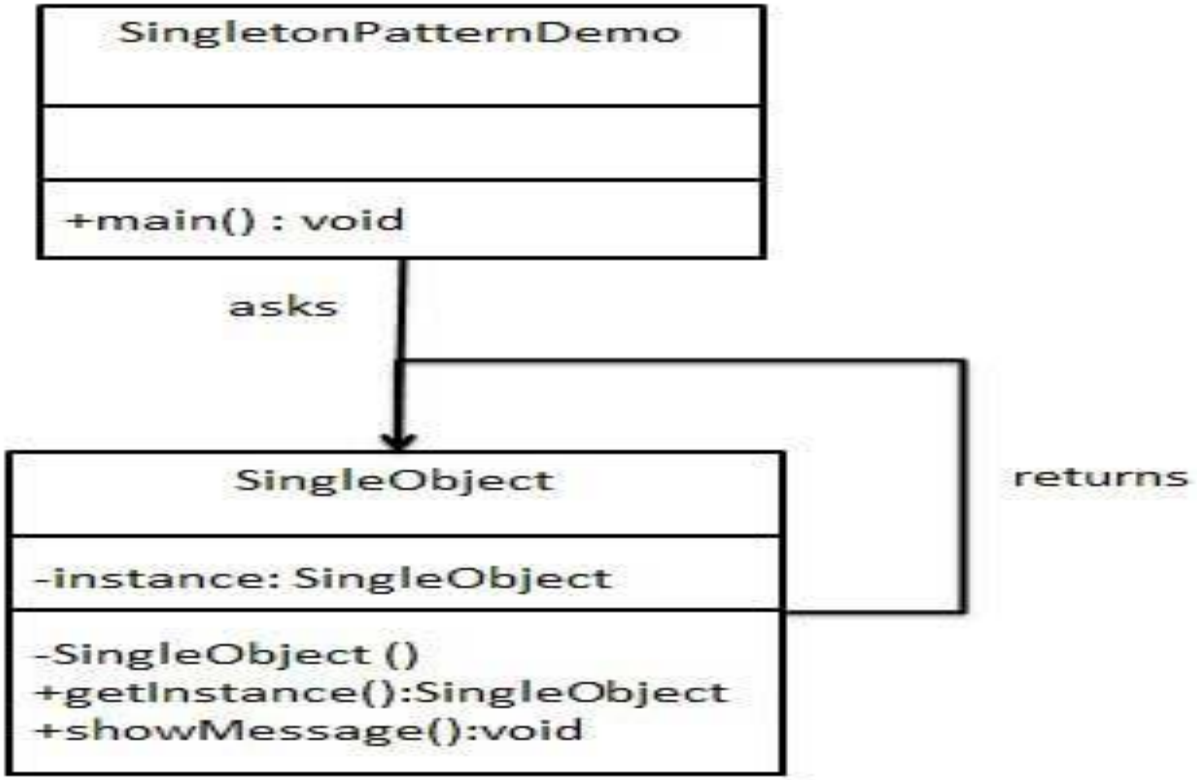
There are two forms of singleton design pattern

Early Instantiation: creation of instance at load time.
Lazy Instantiation: creation of instance when required.

Advantage of Singleton design pattern
Saves memory because object is not created at each request. Only single instance is reused again and again.

Usage of Singleton design pattern
Singleton pattern is mostly used in multi-threaded and database applications. It is used in logging, caching, thread pools, configuration settings etc.

Class Diagram



MVC Design Pattern

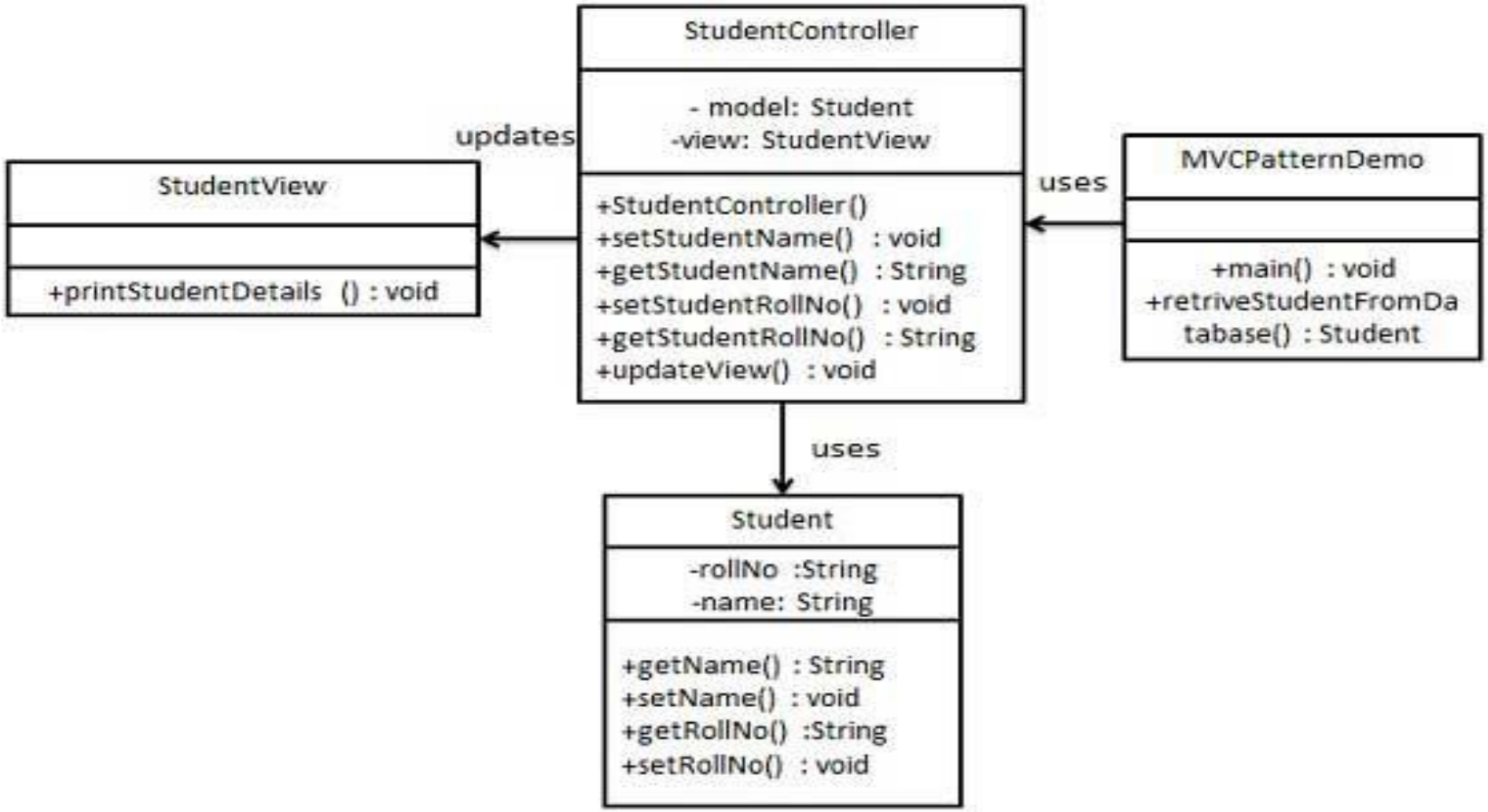
MVC Pattern stands for Model-View-Controller Pattern. This pattern is used to separate application's concerns.

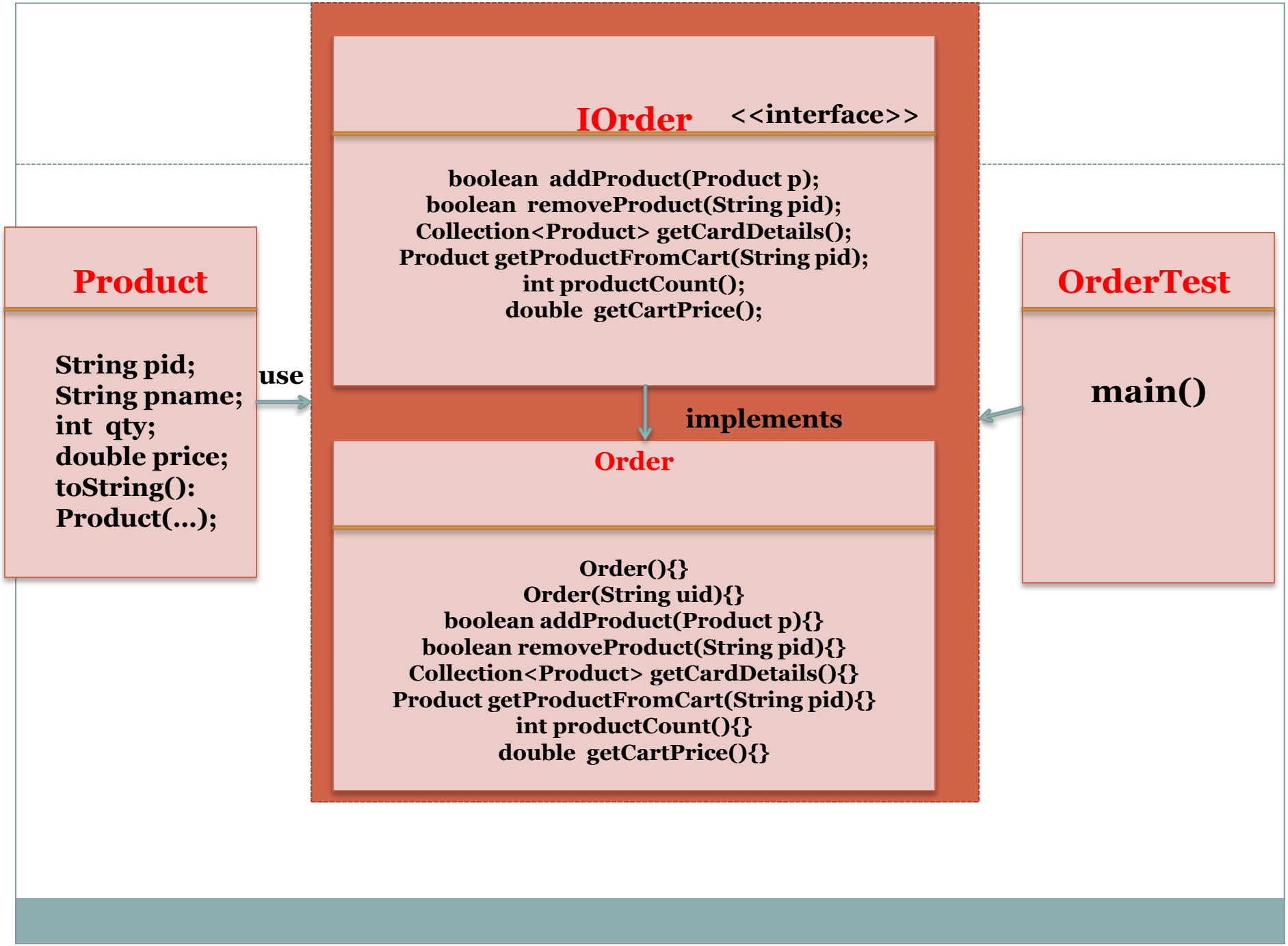
Model - Model represents an object or JAVA POJO carrying data. It can also have logic to update controller if its data changes.

View - View represents the visualization of the data that model contains.

Controller - Controller acts on both model and view. It controls the data flow into model object and updates the view whenever data changes. It keeps view and model separate.

Class Diagram




IOrder
<<interface>>
boolean addProduct(Product p);
boolean removeProduct(String pid);
Collection<Product> getCardDetails();
Product getProductFromCart(String pid);
int productCount();
double getCartPrice();
Product
String pid;
String pname;
int qty;
double price;
toString():
Product(...);
use
implements
Order
Order(){}
Order(String uid){}
boolean addProduct(Product p){}
boolean removeProduct(String pid){}
Collection<Product> getCardDetails(){}
Product getProductFromCart(String pid){}
int productCount(){}
double getCartPrice(){}
OrderTest
main()